



Sistema de Recomendação em Grafos com PLN

Como recomendar livros de forma inteligente, unindo similaridade de conteúdo textual e comportamento de leitores — usando grafos dirigidos, TF-IDF e algoritmos clássicos de ciência da computação.

ESTRUTURAS DE DADOS 2 — FGA0030

UNB · 2026.1

PROF. GLAUCO / LENON

O Problema: Sobrecarga de Informação

Com milhares de livros publicados anualmente, escolher a próxima leitura tornou-se um desafio. Sistemas tradicionais de recomendação dependem de metadados manuais ou de grandes volumes de dados históricos — recursos nem sempre disponíveis em contextos acadêmicos ou projetos independentes.

O Desafio Central

Como recomendar livros de forma inteligente, combinando **similaridade semântica de texto** (via PLN) com o **comportamento de outros leitores**?

A Nossa Solução

Um sistema de recomendação baseado em **grafos dirigidos ponderados**, que extrai semântica das resenhas via TF-IDF e cruza dados de usuários com métricas de grafos clássicas.



Modelagem do Grafo

O sistema utiliza um **digrafo ponderado gerado dinamicamente**, com três tipos de vértices e dois tipos de arestas que codificam tanto o comportamento dos usuários quanto a semântica dos textos.

Vértices — 3 Tipos

USUÁRIO

Leitores cadastrados no sistema

TEXTO

Livros contendo resenhas textuais

GÊNERO

Categorias literárias com palavras-chave

Arestas — 2 Tipos

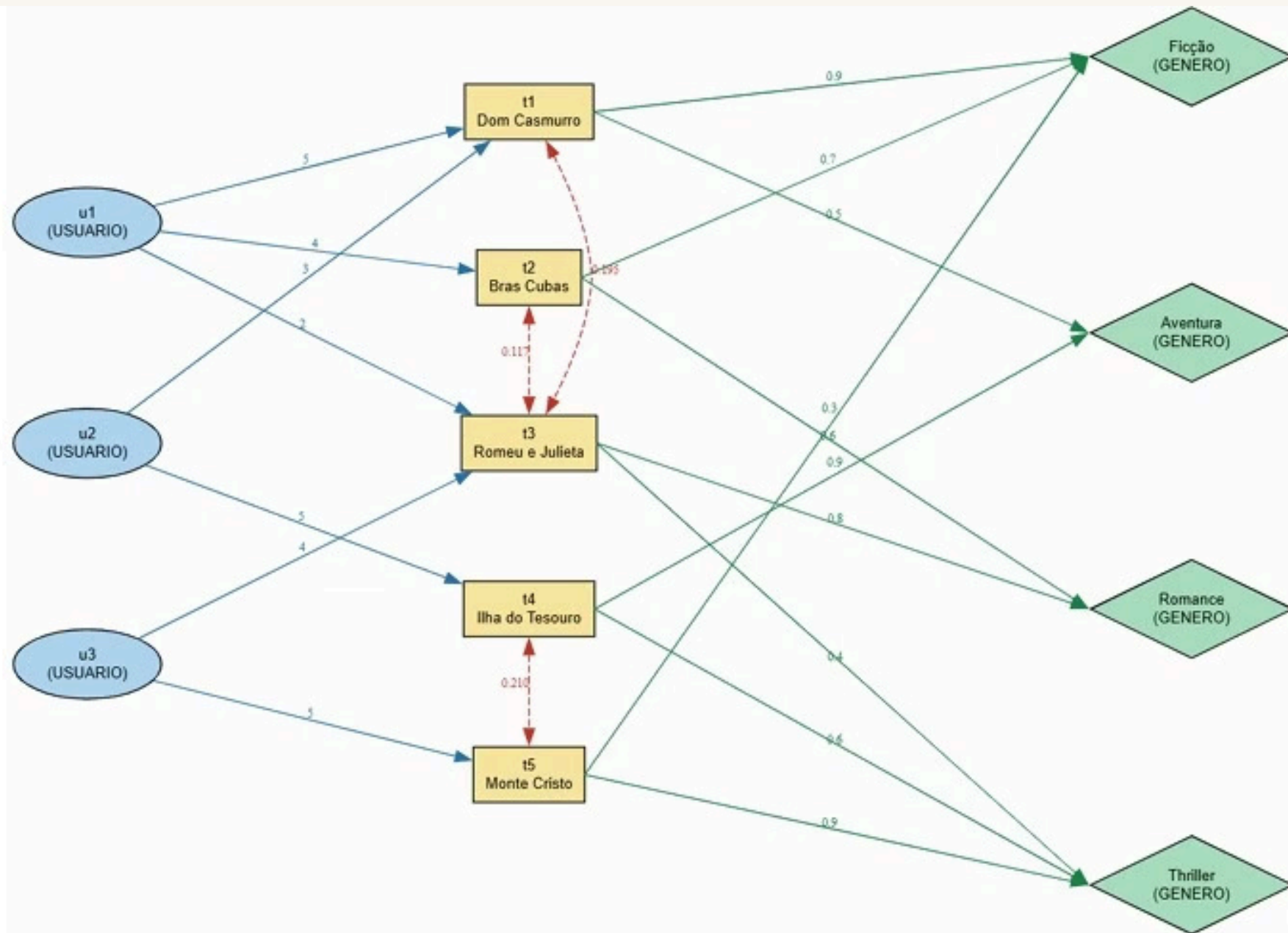
→ Aresta Normal

Usuário → Texto: Peso = Nota (1 a 5)

Texto → Gênero: Peso = Cosseno TF-IDF

→ Projeção Semântica

Texto ↔ Texto: Bidirecional, peso igual à similaridade cosseno entre as resenhas dos livros



Implementação e Estruturas de Dados

O sistema foi desenvolvido **100% em C (C99)**, otimizado com a flag `-O2` e gerenciado via CMake. As estruturas de dados foram escolhidas para garantir eficiência em cada etapa do pipeline.



Grafo — Listas de Adjacência

Implementação nativa do digrafo ponderado, permitindo iteração eficiente sobre vizinhos e inserção dinâmica de arestas durante o processamento PLN.



Tabela Hash — $O(1)$ Amortizado

Desenvolvida do zero com função de espalhamento **djb2** e tratamento de colisões por **sondagem linear**. Usada para indexar o vocabulário de PLN com acesso instantâneo.



Filas (Queues)

Estrutura essencial para o processamento do **algoritmo BFS** (busca em largura) e do **algoritmo de Kahn** (ordenação topológica), ambos centrais no pipeline de recomendação.

O Pipeline de 5 Fases

A solução é organizada em um pipeline sequencial de cinco fases, cada uma responsável por extrair um tipo diferente de sinal de recomendação. O resultado final é um ranking ponderado que combina semântica, comportamento e afinidade de gênero.

Cada fase contribui com uma dimensão distinta para o score final: a **Fase 0** extrai semântica pura das resenhas; as **Fases 1 e 2** exploram a estrutura do grafo de usuários e livros; a **Fase 3** ranqueia gêneros por afinidade; e a **Fase 4** incorpora o filtro colaborativo via similaridade entre usuários.

Fase 0 — PLN: Tokenização, Hash e TF-IDF

A fase de Processamento de Linguagem Natural é a fundação do sistema. Sem intervenção humana, o grafo cria arestas dinâmicas entre livros e gêneros com base puramente no conteúdo textual das resenhas.

01

Tokenização

As resenhas são normalizadas (ASCII, sem acentos) e quebradas em tokens. Pontuação e stopwords são removidas para formar o vocabulário do corpus.

03

Cálculo TF-IDF

Para cada livro, calcula-se o peso de cada termo considerando sua frequência no documento e sua raridade no corpus. Gera vetores esparsos de alta dimensionalidade.

02

Indexação via Hash

Cada token é mapeado para um índice inteiro usando a Tabela Hash com função djb2, garantindo lookup em **$O(1)$ amortizado** durante o cálculo das frequências.

04

Similaridade Cosseno

O cosseno entre vetores TF-IDF cria as arestas **Texto** ↔ **Texto** (projeção semântica) e **Texto** → **Gênero**, conectando livros a categorias temáticas de forma automática.

Fases 1 a 4 — Algoritmos de Grafo

Após a construção do grafo semântico, quatro algoritmos clássicos trabalham em conjunto para filtrar, medir distâncias, ranquear gêneros e identificar comunidades de leitores similares.

1

BFS — Busca de Candidatos

Busca em Largura com **lookup inverso** para encontrar livros não lidos pelo usuário-alvo. Inclui **Fallback Semântico**: caso não haja candidatos diretos, explora vizinhos semânticos via projeção

Texto ↔ Texto.

2

Dijkstra — Distância Personalizada

Algoritmo de Dijkstra com **pesos invertidos**: notas altas (5 estrelas) tornam-se caminhos mais curtos. Livros bem avaliados por usuários similares ficam "mais próximos" no grafo.

3

Kahn — Afinidade de Gênero

Ordenação Topológica (Algoritmo de Kahn) para ranquear os **gêneros favoritos do usuário**. Gêneros com maior fluxo de arestas ponderadas recebem pontuação máxima na recomendação.

4

Jaccard — Filtro Colaborativo

Índice de Jaccard mede a **similaridade entre usuários** com base nas obras em comum. Usuários com Jaccard alto formam uma "bolha" colaborativa que influencia o score final.

Exemplo de Entrada e Saída

Entrada — Dados Sintéticos

Foram cadastrados usuários, notas e resenhas de **23 livros** em ASCII (sem acentos). Exemplo de resenha:

"amor e paixao definem esta historia de forma intensa e marcante"

O sistema exibe no console a **matriz de similaridade cosseno** (texto×texto e texto×gênero) antes de gerar o ranking final.

Saída — Ranking para u1

O sistema exibe o ranking final de recomendação para o usuário-alvo **u1**, detalhando a contribuição de cada uma das 5 fases para cada livro candidato.

Livro t10 — Vencedor

Maior score agregado: Kahn máximo (Romance = gênero favorito), Jaccard 66,6% com a bolha de leitores, e forte ligação TF-IDF com livros já lidos por u1.

Score Detalhado

Cada livro candidato recebe uma nota por fase, permitindo análise de sensibilidade e depuração do impacto de cada algoritmo no resultado final.

Resultados e Escalabilidade

O sistema demonstrou alta eficiência e escalabilidade linear ao ser testado com bases de dados crescentes. O tempo de execução permaneceu estável mesmo com a duplicação do corpus sintético.

23

Livros no Corpus

Resenhas fictícias geradas com coerência temática via LLMs, formatadas em ASCII para compatibilidade com a tokenização em C.

66,6%

Jaccard Máximo

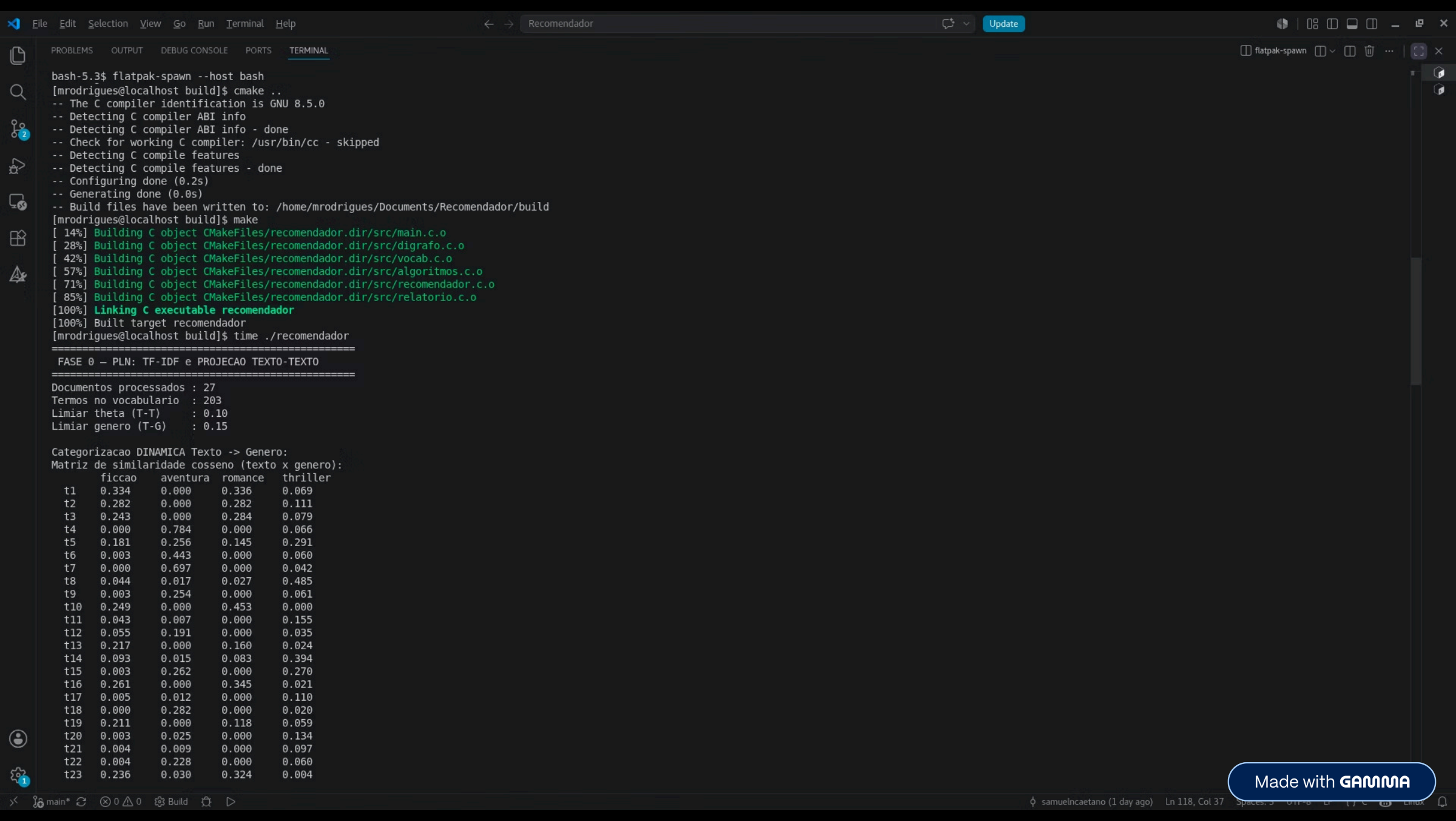
Similaridade entre u_1 e a "bolha" de usuários que leram o livro t_{10} — evidenciando a eficácia do filtro colaborativo.

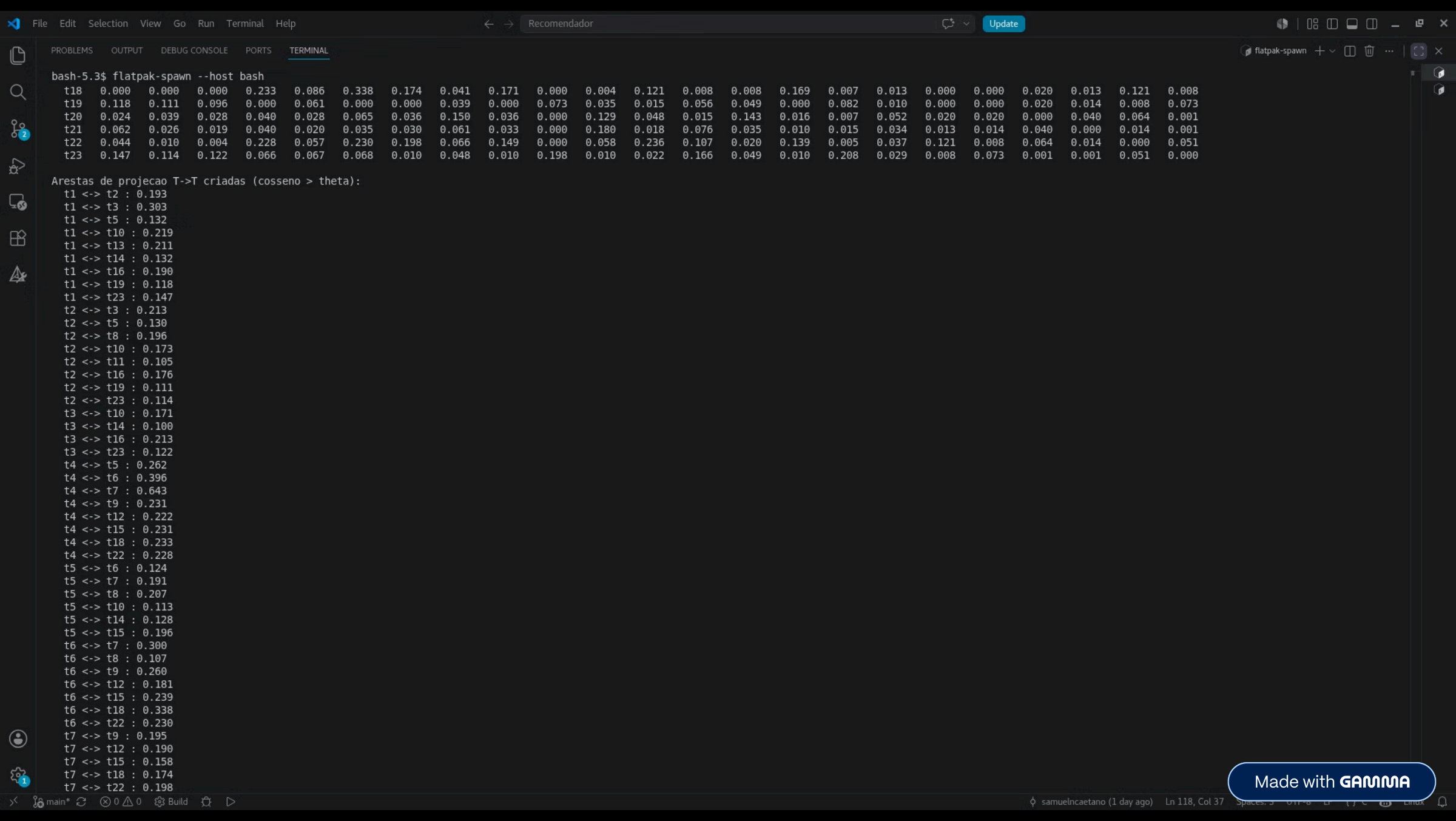
5ms

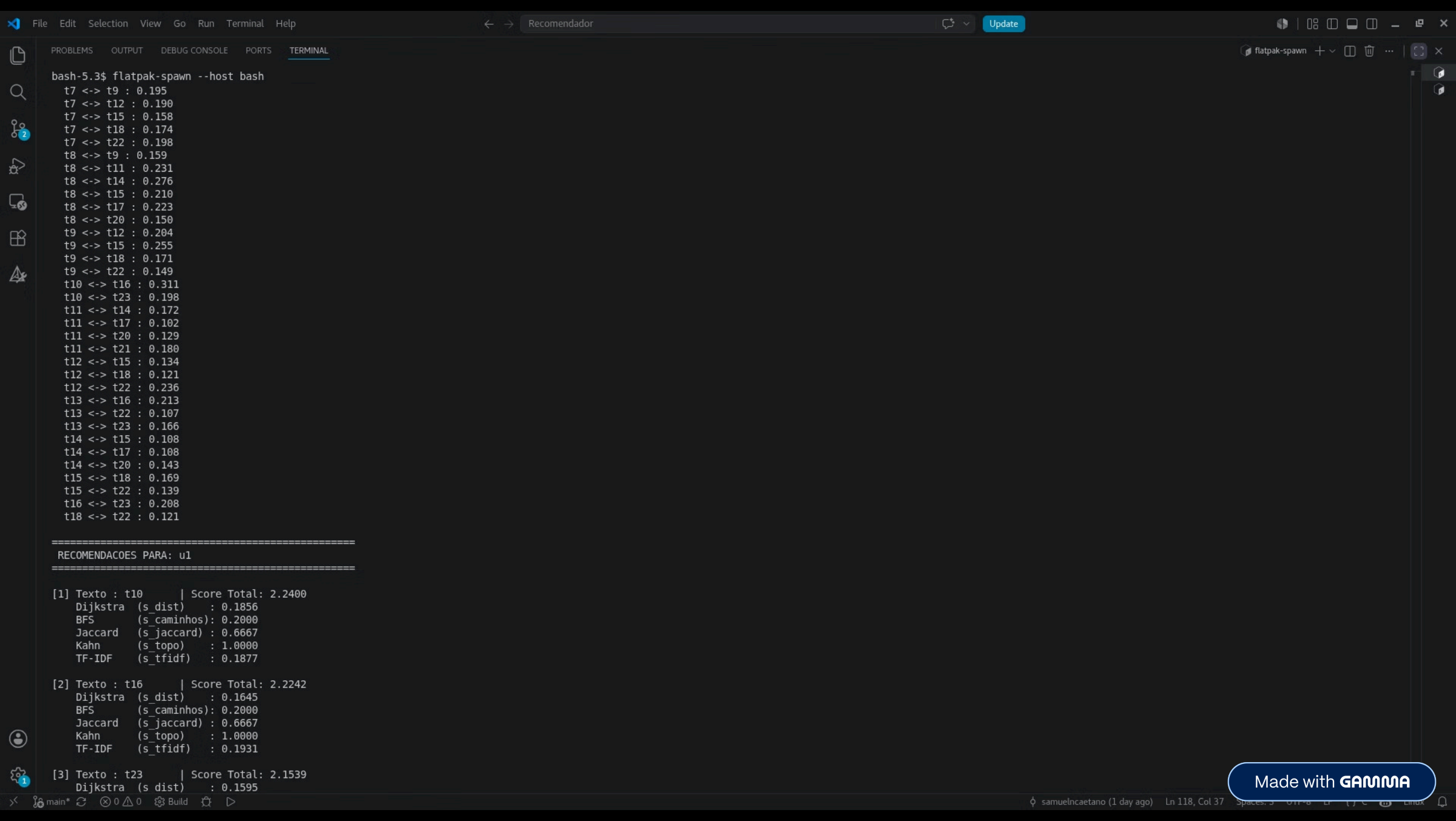
Tempo de Execução

`real 0m0,005s` medido via comando `time` no Linux, mesmo após dobrar a base de dados com novos usuários e livros sintéticos.

- ✓ O comportamento **$O(n)$** foi comprovado empiricamente: ao dobrar a base de dados, o tempo de execução permaneceu praticamente inalterado, confirmando a escalabilidade do pipeline.







```
bash-5.3$ flatpak-spawn --host bash
```

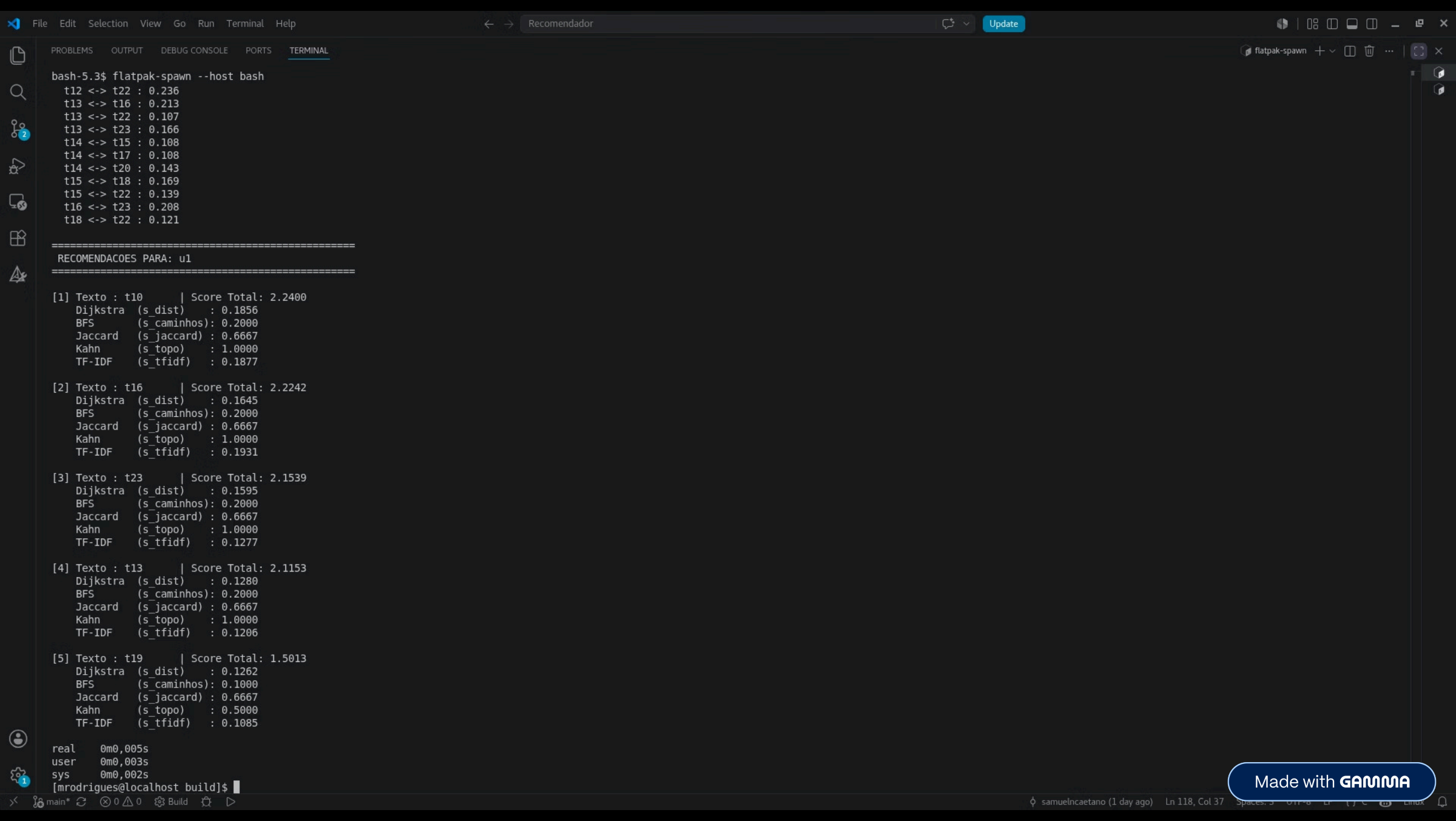
```
t7 <-> t9 : 0.195
t7 <-> t12 : 0.190
t7 <-> t15 : 0.158
t7 <-> t18 : 0.174
t7 <-> t22 : 0.198
t8 <-> t9 : 0.159
t8 <-> t11 : 0.231
t8 <-> t14 : 0.276
t8 <-> t15 : 0.210
t8 <-> t17 : 0.223
t8 <-> t20 : 0.150
t9 <-> t12 : 0.204
t9 <-> t15 : 0.255
t9 <-> t18 : 0.171
t9 <-> t22 : 0.149
t10 <-> t16 : 0.311
t10 <-> t23 : 0.198
t11 <-> t14 : 0.172
t11 <-> t17 : 0.102
t11 <-> t20 : 0.129
t11 <-> t21 : 0.180
t12 <-> t15 : 0.134
t12 <-> t18 : 0.121
t12 <-> t22 : 0.236
t13 <-> t16 : 0.213
t13 <-> t22 : 0.107
t13 <-> t23 : 0.166
t14 <-> t15 : 0.108
t14 <-> t17 : 0.108
t14 <-> t20 : 0.143
t15 <-> t18 : 0.169
t15 <-> t22 : 0.139
t16 <-> t23 : 0.208
t18 <-> t22 : 0.121
```

```
=====
RECOMENDACOES PARA: u1
=====
```

- ```
[1] Texto : t10 | Score Total: 2.2400
 Dijkstra (s_dist) : 0.1856
 BFS (s_caminhos): 0.2000
 Jaccard (s_jaccard) : 0.6667
 Kahn (s_topo) : 1.0000
 TF-IDF (s_tfidf) : 0.1877

[2] Texto : t16 | Score Total: 2.2242
 Dijkstra (s_dist) : 0.1645
 BFS (s_caminhos): 0.2000
 Jaccard (s_jaccard) : 0.6667
 Kahn (s_topo) : 1.0000
 TF-IDF (s_tfidf) : 0.1931

[3] Texto : t23 | Score Total: 2.1539
 Dijkstra (s_dist) : 0.1595
```



# Equipe e Contribuições

Projeto desenvolvido em C99 com uso restrito de IA conforme as regras da disciplina — apenas para geração de dados sintéticos e apoio ao refatoramento.



## Samuel

Arquitetura base do grafo e modelagem das **Fases 1 a 4**: implementação de Dijkstra, Kahn, BFS e Índice de Jaccard.



## Thiago

Modelagem do **pipeline de PLN (Fase 0)**: tokenização, cálculo do TF-IDF e métricas de similaridade cosseno entre textos e gêneros.



## Letícia

Criação dos slides, **testes de estresse de complexidade**, Revisão do código, expansão do banco de dados sintético, Criação dos Slides e Imagens(Claude/Gemini).



## Marjorie

Expansão de Dados: Adição do usuário `u9` e 3 novos livros com resenhas sintéticas geradas por IA.

Refatoração: Dobrou a capacidade da Tabela Hash (`MAX_VOCAB = 1024`) para suportar o novo volume no TF-IDF sem falhas.

Escalabilidade final: Comprovou a eficiência da arquitetura medindo o tempo de execução via comando `time` (0m0,005s).



## Gabriela

Atuou em par (Pair Programming) com Letícia e Marjorie na expansão do banco sintético e na refatoração da Tabela Hash (`MAX_VOCAB = 1024`), assegurando o suporte ao novo volume de dados no TF-IDF.

**Apresentação e Síntese:** Estruturação lógica, design e revisão técnica dos slides de apresentação.